# 🛠️ Stage 0: Environment & Repository Setup — DocDue

> **Goal:** Have a fully initialized monorepo on GitHub with the correct folder structure, Git branch strategy, virtual environments, and all tooling installed — so every sprint starts from a clean, professional foundation.

## 📋 Overview of This Stage

| Step | Activity |
|---|---|
| 0.1 | Prerequisites — install core tools |
| 0.2 | Create the monorepo folder structure |
| 0.3 | Initialize Git and push to GitHub |
| 0.4 | Set up the Python (FastAPI) backend environment |
| 0.5 | Set up the Next.js frontend environment |
| 0.6 | Create `.env.example` files (safe credential templates) |
| 0.7 | Update `.gitignore` to cover everything |
| 0.8 | Create the Git branch strategy |
| 0.9 | Set up GitHub Project Board (Scrum tracking) |
| 0.10 | Final verification checklist |

## ☑️ 0.1 — Prerequisites (Install These First)

Before touching any code, confirm these tools are installed on your machine.

### 🔧 Tool Checklist

| Tool | Why You Need It | Check If Installed |
|---|---|---|
| **Python 3.11+** | Backend runtime (FastAPI, OCR) | `python --version` |
| **Node.js 20+ (LTS)** | Frontend runtime (Next.js) | `node --version` |
| **npm** | Package manager for frontend | `npm --version` |
| **Git** | Version control | `git --version` |
| **Tesseract OCR** | OCR engine (installed system-wide) | `tesseract --version` |
| **VS Code** | IDE | Already using it ☑️ |

#### Install Tesseract OCR (Windows)

Tesseract must be installed **system-wide** (not via pip).

1. Download the installer from: https://github.com/UB-Mannheim/tesseract/wiki
2. Run the `.exe` installer (pick the latest stable version).
3. During install, **check the box** for "Add to PATH" and also install **English language data**.
4. Verify after install:

```
tesseract --version
```

5. Note down the install path (usually `C:\Program Files\Tesseract-OCR\tesseract.exe`) — you'll need it in your `.env` file later.

---

## 0.2 — Create the Monorepo Folder Structure

Your repo uses a **monorepo** layout: one Git repo containing both `frontend/` and `backend/` folders.

Open a terminal in your project folder `d:\5TH-TRIME\D-PROJECTZ\DOCDUE\DocDue` and run:

```
# Create the main subdirectories
mkdir backend
mkdir frontend

# Create backend subfolders
mkdir backend\app
mkdir backend\app\api
mkdir backend\app\api\routes
mkdir backend\app\core
mkdir backend\app\models
mkdir backend\app\schemas
mkdir backend\app\services
mkdir backend\app\workers
mkdir backend\tests

# Create placeholder __init__.py files so Python treats these as packages
New-Item backend\app\__init__.py -ItemType File
New-Item backend\app\api\__init__.py -ItemType File
New-Item backend\app\api\routes\__init__.py -ItemType File
New-Item backend\app\core\__init__.py -ItemType File
New-Item backend\app\models\__init__.py -ItemType File
New-Item backend\app\schemas\__init__.py -ItemType File
New-Item backend\app\services\__init__.py -ItemType File
New-Item backend\app\workers\__init__.py -ItemType File
New-Item backend\tests\__init__.py -ItemType File

# Create a backend entry-point file
New-Item backend\main.py -ItemType File
```

### Final Folder Structure (What You're Building)

```
DocDue/                          ← Repo root (already exists)
├── .git/                        ← Git internals (auto-created)
├── .gitignore                   ← Already exists, will update
├── .env.example                 ← Safe template (NO real secrets)
├── CLAUDE.md                    ← Project context (already exists)
├── README.md                    ← Will update this
│
├── backend/                     ← FastAPI backend (Python)
│   ├── main.py                  ← App entry point
│   ├── requirements.txt         ← Python dependencies
│   ├── .env.example             ← Backend env template
│   ├── .venv/                   ← Virtual environment (gitignored)
│   └── app/
│       ├── __init__.py
│       ├── api/
│       │   ├── __init__.py
│       │   └── routes/          ← Route handlers (auth, docs, etc.)
│       │       └── __init__.py
│       ├── core/                ← Config, security, JWT
│       ├── models/              ← Beanie ODM MongoDB document models
│       ├── schemas/             ← Pydantic request/response schemas
│       ├── services/            ← Business logic (OCR, S3, email)
│       └── workers/             ← APScheduler background jobs
│
├── frontend/                    ← Next.js frontend (TypeScript)
│   ├── package.json
│   ├── next.config.js
│   ├── tailwind.config.js
│   ├── .env.example             ← Frontend env template
│   └── src/
│       ├── app/                 ← Next.js App Router pages
│       ├── components/          ← Reusable UI components
│       ├── lib/                 ← API client, utilities
│       └── types/               ← TypeScript type definitions
│
└── tests/                       ← Backend tests live under backend/tests/
```

---

## 🔀 0.3 — Initialize Git & Push to GitHub

Your repo already has a Git history. Here's how to connect it to a **new GitHub repository** and push everything up.

### Step 1: Create the GitHub Repository

1. Go to https://github.com/new
2. Set **Repository name:** `DocDue`
3. Set **Visibility:** Private (recommended for a student project with credentials)
4. **DO NOT** initialize with README, .gitignore, or license (your repo already has these).
5. Click **"Create repository"**.

6. Copy the remote URL shown (e.g., `https://github.com/YOUR_USERNAME/DocDue.git`).

### Step 2: Connect Your Local Repo to GitHub

Run this in your project root (`d:\5TH-TRIME\D-PROJECTZ\DOCDUE\DocDue`):

```
# Check current git status
git status

# Check if a remote already exists
git remote -v

# If NO remote exists, add one (replace URL with yours from GitHub)
git remote add origin https://github.com/YOUR_USERNAME/DocDue.git

# If a remote already exists but is wrong, update it
git remote set-url origin https://github.com/YOUR_USERNAME/DocDue.git
```

### Step 3: Make Your First Commit

```
# Stage everything
git add .

# Commit with a descriptive message
git commit -m "chore: initial project setup — monorepo structure, CLAUDE.md,
.gitignore"

# Set the default branch to 'main' (best practice)
git branch -M main

# Push to GitHub for the first time
git push -u origin main
```

> [!IMPORTANT] If `git push` asks for credentials, use your GitHub username and a **Personal Access Token (PAT)** as the password (not your GitHub account password). Create one at: https://github.com/settings/tokens → Generate new token (classic) → check `repo` scope.

---

## 0.4 — Backend: Python Virtual Environment & Dependencies

### Step 1: Create and Activate Virtual Environment

```
# Navigate into backend folder
cd backend

# Create the virtual environment
python -m venv .venv
```

```
# Activate it (Windows PowerShell)
.venv\Scripts\Activate.ps1

# You should see (.venv) prefix in your terminal now
```

> [!NOTE] If PowerShell blocks the activation script, run this once: `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`

### Step 2: Install All Required Python Packages

```
# Upgrade pip first
python -m pip install --upgrade pip

# Install all dependencies (MongoDB Motor driver + Beanie ODM)
pip install fastapi uvicorn[standard] motor beanie pymongo python-dotenv python-jose[cryptography] passlib[bcrypt] python-multipart boto3 pytesseract Pillow pdf2image apscheduler

# Install dev/test tools
pip install pytest httpx pytest-asyncio
```

### Step 3: Save Dependencies to requirements.txt

```
pip freeze > requirements.txt
```

This creates a `requirements.txt` file so anyone can recreate the exact environment later with:

```
pip install -r requirements.txt
```

### What Each Package Does

| Package | Purpose |
|---|---|
| `fastapi` | Web framework for the API |
| `uvicorn[standard]` | ASGI server to run FastAPI |
| `motor` | Async Python driver for MongoDB |
| `beanie` | Async ODM (Object-Document Mapper) for MongoDB based on Pydantic |
| `pymongo` | Standard MongoDB driver |
| `python-dotenv` | Loads `.env` file into environment variables |

| Package | Purpose |
|---|---|
| `python-jose[cryptography]` | JWT token generation and validation |
| `passlib[bcrypt]` | Password hashing with bcrypt |
| `python-multipart` | Parses file uploads (multipart form data) |
| `boto3` | AWS SDK — for interacting with S3 |
| `pytesseract` | Python wrapper for Tesseract OCR engine |
| `Pillow` | Image processing library (used with pytesseract) |
| `pdf2image` | Converts PDF pages to images for OCR |
| `apscheduler` | Background scheduler (daily expiry scan job) |
| `pytest` + `httpx` | Testing framework + async HTTP client for tests |

### Step 4: Create a Minimal `main.py` to Verify Setup

Open `backend/main.py` and paste this:

```
from fastapi import FastAPI

app = FastAPI(
    title="DocDue API",
    description="Document expiry tracking SaaS — backend API",
    version="0.1.0"
)

@app.get("/")
def health_check():
    return {"status": "ok", "message": "DocDue API is running"}
```

### Step 5: Run the Server to Verify It Works

```
# Make sure you're in the backend/ folder with (.venv) active
uvicorn main:app --reload
```

Open your browser to `http://127.0.0.1:8000` — you should see:

```
{"status": "ok", "message": "DocDue API is running"}
```

Also check the auto-generated docs at: `http://127.0.0.1:8000/docs`

Press `Ctrl+C` to stop the server.

```
# Go back to repo root
cd ..
```

---

## 0.5 — Frontend: Next.js Setup

```
# Navigate to repo root first
cd d:\5TH-TRIME\D-PROJECTZ\DOCDUE\DocDue

# Scaffold Next.js inside the frontend/ folder
npx create-next-app@latest frontend --typescript --tailwind --eslint --app --src-
dir --import-alias "@/*" --use-npm
```

When prompted:

- **TypeScript?** → Yes
- **ESLint?** → Yes
- **Tailwind CSS?** → Yes
- `src/` **directory?** → Yes
- **App Router?** → Yes
- **Import alias?** → Yes, use `@/*`

### Verify the Frontend Works

```
cd frontend
npm run dev
```

Open `http://localhost:3000` — you should see the default Next.js welcome page.

Press `Ctrl+C` to stop.

```
# Go back to repo root
cd ..
```

---

## 0.6 — Create `.env.example` Files (Safe Templates)

These files are committed to Git. They show what variables are needed WITHOUT any real secret values.

### Root-Level `.env.example`

Create `DocDue/.env.example`:

```
# ======================================
# DocDue — Environment Variables Template
# ======================================
# Copy this file to .env and fill in real values.
# NEVER commit .env to Git.

# --- Database (Supabase PostgreSQL) ---
DATABASE_URL=postgresql://user:password@host:5432/docdue_db

# --- JWT Auth ---
SECRET_KEY=your-super-secret-jwt-key-here
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
REFRESH_TOKEN_EXPIRE_DAYS=7

# --- AWS S3 ---
AWS_ACCESS_KEY_ID=your-aws-access-key
AWS_SECRET_ACCESS_KEY=your-aws-secret-key
AWS_REGION=ap-south-1
S3_BUCKET_NAME=docdue-documents

# --- Tesseract OCR ---
TESSERACT_PATH=C:\Program Files\Tesseract-OCR\tesseract.exe

# --- SMTP Email (Gmail or Resend) ---
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USERNAME=your-email@gmail.com
SMTP_PASSWORD=your-app-password

# --- App Settings ---
ENVIRONMENT=development
FRONTEND_URL=http://localhost:3000
```

### Backend `.env.example`

Create `DocDue/backend/.env.example` with the same content (the backend loads its own `.env`).

### Frontend `.env.example`

Create `DocDue/frontend/.env.example`:

```
# ======================================
# DocDue Frontend — Environment Variables
# ======================================

# Backend API URL
NEXT_PUBLIC_API_URL=http://localhost:8000
```

## 🚫 0.7 — Update `.gitignore`

Your existing `.gitignore` is good. Let's enhance it to cover all project-specific needs.

Replace the content of `.gitignore` with:

```
# ====================================
# DocDue — .gitignore
# ====================================

# --- Environment files (NEVER commit these) ---
.env
.env.local
.env.*.local
backend/.env
frontend/.env
frontend/.env.local

# --- Python / Backend ---
__pycache__/
*.py[cod]
*$py.class
*.egg-info/
.venv/
venv/
env/
.pytest_cache/
*.sqlite3
.mypy_cache/
.ruff_cache/
dist/
build/
*.egg

# --- Alembic (keep structure, ignore local cache) ---
# alembic/versions/ is committed on purpose

# --- Node.js / Frontend ---
node_modules/
.next/
out/
.npm/
.eslintcache

# --- Testing & Coverage ---
coverage/
.coverage
htmlcov/
*.lcov

# --- Secrets & Keys ---
*.pem
```

```
*.key
*.p12
*.pfx

# --- OS Files ---
.DS_Store
Thumbs.db
desktop.ini

# --- Editor ---
.vscode/settings.json
*.suo
*.user
*.swp
*.swo
*~

# --- Build artifacts ---
/build
*.tsbuildinfo
next-env.d.ts

# --- Logs ---
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# --- Vercel / Deployment ---
.vercel
```

---

## 🌿 0.8 — Git Branch Strategy

This project uses a **feature-branch workflow** aligned with the sprint plan.

### Create All Feature Branches Now

```
# Make sure you're on main first
git checkout main

# Create all sprint branches upfront
git checkout -b feature/sprint1-auth-db
git checkout main

git checkout -b feature/sprint2-s3-upload
git checkout main

git checkout -b feature/sprint3-ocr-pipeline
git checkout main
```

```
git checkout -b feature/sprint4-scheduler-alerts
git checkout main

git checkout -b feature/sprint5-frontend-dashboard
git checkout main

git checkout -b feature/sprint6-deployment
git checkout main

# Push all branches to GitHub
git push origin feature/sprint1-auth-db
git push origin feature/sprint2-s3-upload
git push origin feature/sprint3-ocr-pipeline
git push origin feature/sprint4-scheduler-alerts
git push origin feature/sprint5-frontend-dashboard
git push origin feature/sprint6-deployment

# Confirm you are back on main
git branch
```

### Branch Rules (Follow These Always)

| Branch | Purpose |
|---|---|
| `main` | Always stable — only receives **merged, tested** code |
| `feature/sprint1-auth-db` | Sprint 1 work: Auth + DB schema |
| `feature/sprint2-s3-upload` | Sprint 2 work: AWS S3 integration |
| `feature/sprint3-ocr-pipeline` | Sprint 3 work: Tesseract OCR pipeline |
| `feature/sprint4-scheduler-alerts` | Sprint 4 work: APScheduler + SMTP |
| `feature/sprint5-frontend-dashboard` | Sprint 5 work: Next.js frontend |
| `feature/sprint6-deployment` | Sprint 6 work: Deployment configs |

**Workflow for each sprint:**

```
# 1. Switch to the sprint branch before working
git checkout feature/sprint1-auth-db

# 2. Make changes, then commit frequently
git add .
git commit -m "feat(auth): add JWT token generation utility"

# 3. Push your branch to GitHub
git push origin feature/sprint1-auth-db

# 4. When sprint is DONE and tested, merge back to main
git checkout main
```

```
git merge feature/sprint1-auth-db
git push origin main
```

---

## 📊 0.9 — GitHub Project Board (Scrum Tracking)

This is important for your **project report** — it provides evidence of Agile/Scrum methodology.

### Steps to Set Up

1. Go to your GitHub repo → click the **"Projects"** tab.
2. Click **"Link a project"** → **"New project"**.
3. Choose **"Board"** view (Kanban-style).
4. Name it: `DocDue — Sprint Board`.
5. Create these **columns**:
   - 📋 `Backlog`
   - 🚧 `In Progress`
   - 👀 `In Review`
   - ☑️ `Done`
6. Add issues for each sprint (example for Sprint 1):
   - `[Sprint 1] Design & create PostgreSQL schema`
   - `[Sprint 1] Implement user registration endpoint`
   - `[Sprint 1] Implement login + JWT token issue`
   - `[Sprint 1] Implement token refresh endpoint`
   - `[Sprint 1] Password hashing with bcrypt`
   - `[Sprint 1] Tenant isolation middleware`

> [!TIP] Move cards across columns as you work. Screenshots of the board in motion make excellent evidence for your project report viva.

---

## 🚀 0.10 — Commit the Setup & Final Git Commands

Now commit everything you've set up in this stage.

```
# Make sure you're on main
git checkout main

# Stage all new files
git add .

# Check what you're about to commit (sanity check)
git status

# Commit the full setup
git commit -m "chore: complete Stage 0 — monorepo structure, venv, Next.js
scaffold, .env.example, updated .gitignore"
```

```
# Push to GitHub
git push origin main
```

Tag This Milestone (Optional but Recommended for Report)

```
# Create an annotated tag for Stage 0 completion
git tag -a v0.1-setup -m "Stage 0 complete: environment and repo fully
initialized"

# Push the tag to GitHub
git push origin v0.1-setup
```

---

## ☑ 0.10 — Final Verification Checklist

Before calling this stage done, verify every item below:

```
[ ] python --version           → shows 3.11+
[ ] node --version             → shows 20+
[ ] tesseract --version        → shows version number
[ ] git remote -v              → shows your GitHub repo URL

[ ] backend/.venv/ exists      → virtual environment created
[ ] backend/requirements.txt   → all packages listed
[ ] uvicorn main:app --reload  → server starts, http://localhost:8000 responds
[ ] http://localhost:8000/docs → FastAPI Swagger UI loads

[ ] frontend/package.json      → Next.js project initialized
[ ] npm run dev (in frontend/) → starts at http://localhost:3000

[ ] .env.example               → exists at root, backend/, and frontend/
[ ] .env files                 → NOT showing in git status (gitignored)
[ ] .gitignore                 → updated and committed

[ ] git log --oneline          → shows at least 1 commit on main
[ ] git branch -a              → all 6 feature branches visible
[ ] GitHub repo                → branches visible online
[ ] GitHub Project Board       → created with sprint columns
[ ] git tag                    → v0.1-setup tag pushed to GitHub
```

---

## 🚀 What's Next: Sprint 1

With your environment ready, **Sprint 1** will cover:

- Designing and creating all MongoDB Document Collections on **MongoDB Atlas**
- Building the **FastAPI Auth system**: registration, login, JWT issue & refresh, bcrypt hashing

- Setting up **Beanie ODM & Motor** database connection initialization
- Implementing **tenant isolation middleware**

---

*Document created for DocDue — MCA Final Project, CHRIST (Deemed to be University) Developer: Dinu Devees George*